



Ontologies UPM

Self-Optimizing Ontology Project

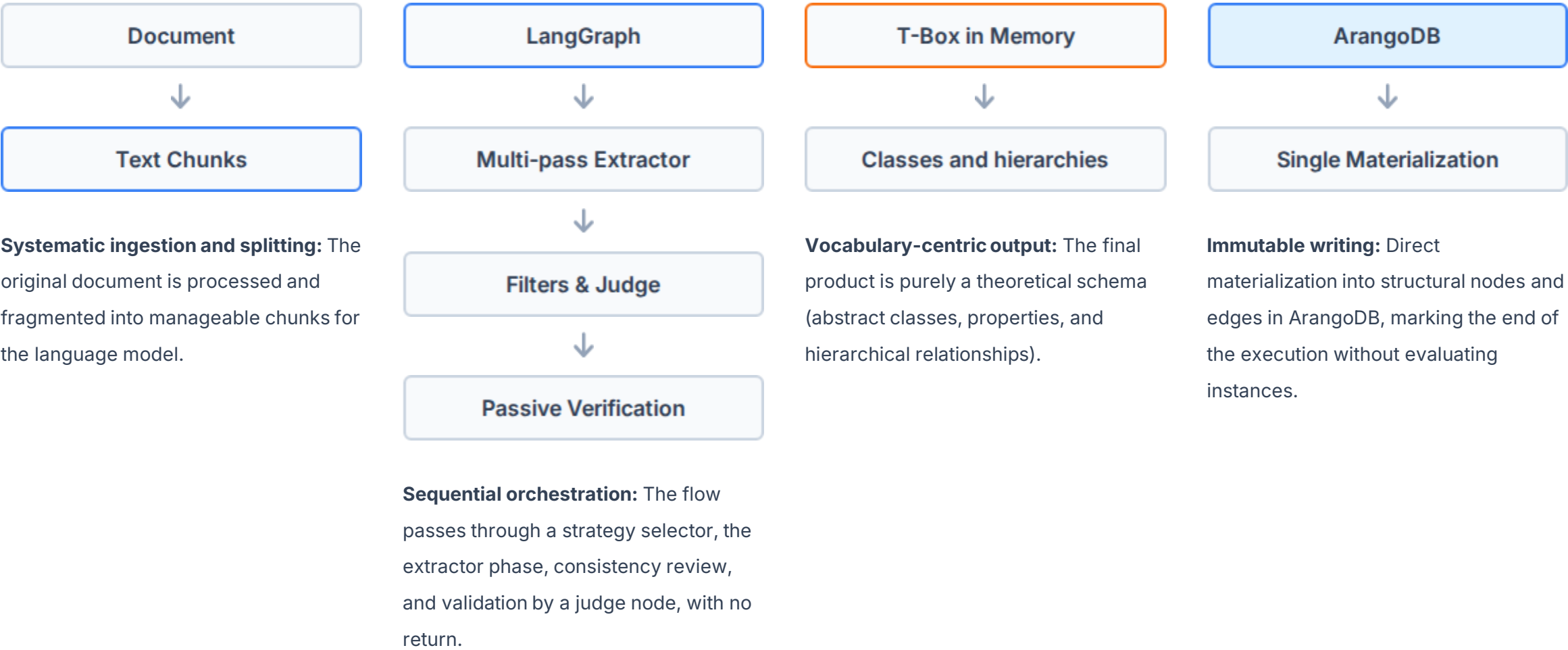


Pipeline 1: Linear Extraction

(Original)

Original baseline focused exclusively on the schema (T-box) without correction iterations.

Pipeline 1 Flow (Baseline)



Critical Limitations of the Linear Pipeline



Schema Vulnerability

Being an effective single-pass design, there is an absolute dependence on the LLM's first response. There is no structured error feedback.

Direct consequence: if the model returns a disconnected schema, with orphaned or excessively generic classes, it is inevitably materialized and compromises the entire database.



Incapacity for Multihop

A structured second phase oriented to the data layer or "A-box" (named individuals and their direct relationships) is not executed.

Since there are no systematically linked instances, it becomes impossible to resolve practical domain queries, turning the system into a mere generator of isolated taxonomies.



Pipeline 2: Refinement Loop

Introduction of the iterative "Loop 1" to diagnose and repair the T-box structure using rules and LLMs.

Quality Loop Flow (T-box)



Rigorous evaluation

quality_gate_aggregate is implemented to measure connectivity, structural integrity, and semantic coverage against the base text.



Deterministic repairs

If the gates fail, non-LLM rules are executed to link orphans and complete minimum mandatory object properties.



Surgeon Agent

If the schema remains deficient, an optimizer_surgeon injects bounded patches over the damaged subgraph.



Controlled iteration

The cycle returns to the judge until passing the tests or exhausting the attempts (~3), only then proceeding to final materialization.

Pipeline 2 Results

📈 Evolution Achievements

Proactive detection: The system now identifies the lack of critical classes or properties before validating the work, avoiding dumping broken schemas.

Hybrid effort: The combination of deterministic rules, which are 100% reliable, alongside an LLM explicitly guided by the error report, generates highly coherent schemas.

⚠️ Limitations

Loss by exhaustion: If loop 1 exhausts attempts without passing gates, the run ends without materializing the T-box.

The instance abyss: Even if the schema is robust, we still lack a systematic process to extract, verify, and reconnect the individuals extracted from the text.



Pipeline 3: Double Loop

Separation of responsibilities: Materialized T-box followed by an iterative second loop dedicated exclusively to the data layer (A-box).

Definitive Pipeline Global Flow



Phase A: Loop 1

Stabilize Schema

Phases A and B guarantee that the structural vocabulary is formalized and immutable in ArangoDB before processing entities.

Phase B: Materialize T-Box

Fixed Schema in DB

Model anchoring ensures stability before moving to instance data.

Phase C: Loop 2 (A-Box)

Iterative Instances

For each attempt, specialized extraction guided by the actual T-box ensures individuals obey the model. The A-box is cleanly rebuilt; each attempt purges previous instances.

▶▶▶ Extra Operation

(Rerun)

If a T-box in Arango is valid but instances failed, it is possible to relaunch only Phase C, saving latency and computational costs.

Internal Anatomy of Loop 2 (A-box)



Extraction and Relationships

The LLM extracts individuals by grouping chunks, attaching the actual T-box in the prompt. Then, a second exclusive LLM pass generates cross-relations using a closed catalog of entities, preventing it from inventing names that break links.



Automatic Repairs

Before storing, critical deterministic actions are executed: URI normalization to fit the schema, semantic re-typing of misclassified instances, and rigorous filtering of false positives (like mistakenly classified quotes or long phrases).



Materialization and Feedback

Instances are written and evaluated through specific quality gates. If these fail, the system compiles precise reports that are transformed into textual instructions ("Refinement Feedback") injected into the LLM in the next retry.

How does Loop 2 diagnose errors?



Evaluation without re-reading

Validations DO NOT re-process the original document looking for lost answers.



State comparison

Directly compares the generated JSON against the strict ArangoDB schema and the actual result of the graph write.



Post-write metrics

Audits exact counts: omitted nodes (skipped), dropped relationships due to invalid identifiers (dropped), and the materialized/declared ratio.



"Island" detection

Penalizes the presence of correctly typed instances that have zero edges connecting to other entities.

Active Correction and Architecture Limits

Correction Mechanisms

Co-occurrence inference: If the LLM omits links, retry rules force relationships based on co-occurrence in the same text chunks.

Explicit Prompt Feedback: Errors are converted into literal commands on retry: "Relation X failed: unknown target; use the exact name from the list".

Limitations

Double generative dependence: Multiple LLM calls are required in both loops, increasing total processing time and API costs.

Graph vs Generation: Enabling multihop traceability in the database is an analytical triumph, but it doesn't equate to getting perfect verbal chat answers to any ambiguous question.

Architectural Evolution Comparison

Analysis Criterion	Pipeline 1 (Baseline)	Pipeline 2 (T-box Loop)	Pipeline 3 (Double Loop)
Materialization	Only T-box	Only validated T-box	Fixed T-box + interactive A-box
Error Detection	Non-existent or passive	Gates applied to schema	Schema Gates + A-box Metrics
Corrective Strategy	No retries	Rules and LLM Surgeon	Rules + Double LLM pass + Prompt feedback
Targeted Bottleneck	Starting point	Disconnected schema	Individuals without edges / Broken links
Current Main Limitation	Bad due to rigid design	Weak or absent A-box	Graph ≠ semantics

Extraction Prompts (1/2): Schema (T-box)

#tier1_standard

[System - expanded extract]

You are an expert ontology engineer skilled in OWL 2 and RDFS. Extract a formal domain ontology.
Return valid JSON with this schema:

```
{
  "classes": [{
    "uri": "namespace#ClassName",
    "label": "...",
    "description": "...",
    "parent_uri": "Parent URI | null",
    "properties": [{
      "property_type": "datatype | object",
      "range": "Class URI or xsd:string"
    }]
  }]
}
```

Key Guidelines:

- Identify owl:Classes and rdfs:subClassOf hierarchies
- Object properties: range = EXACT URI of another class within the same JSON
- A single consistent namespace for all classes
- NEVER parent_uri pointing to the class itself
- Extract ALL explicit relationships. DO NOT create properties towards non-extracted classes.

[User]

Extract an OWL ontology from the following chunks: {chunks_text}. Return ONLY valid JSON.

- ✓ Used structurally in Pipeline 1, Pipeline 2, and Pipeline 3 (LangGraph phase).
- 📄 Defines the fundamental vocabulary of the domain **before** extracting any instance.

Extraction Prompts (2/2): Instances (A-box, P3)

Phase 1: Individuals

ABOX_STANDARD

[System] The T-box is already extracted and is FIXED. Extract named individuals.

Fixed T-box (do NOT invent new classes)

- Professor | uri=http://example.org/domain#Professor | prop: name ...
- Subject | uri=... #Subject | prop: credits ...
- Subject --[taughtBy]→ Professor

JSON:

```
{ "individuals": [{  
  "label": "canonical name of the text",  
  "class_uri": "EXACT URI of the T-box",  
  "emailAddress": "...",  
  "object_relations": [{  
    "property_label": "taughtBy",  
    "target_label": "Big Data",  
    "target_type": "individual | class"  
  }]  
}] }
```

Rules:

- Only explicit named entities. class_uri ALWAYS from the T-box.
- Person → person-like; DO NOT type long titles as Person.

Phase 2: Relations

ABOX_RELATIONS

[System] Individuals ALREADY extracted. SOLE task: add object_relations.

Closed catalog:

- Prof. Ana López | class: Professor
- Big Data | class: Subject

Object properties from the T-box:

- Professor --[teaches]→ Subject

Output:

```
{  
  "relation_updates": [{  
    "from_label": "...",  
    "object_relations": [...]  
  }]  
}
```

Critical restrictions:

- EXACT labels from the catalog; do not paraphrase.
- Do not invent individuals or properties.

Loop 1: T-box optimization (Surgeon + rules)

👤 surgeon_optimizer

```
[System - extract]
You are a repair assistant. You receive: JSON subgraph, allowed URIs, and failures.
Return A JSON with ONLY the classes that change:
{ "patches": [{
  "uri": "<Class URI from the allowlist>",
  "parent_uri": "<optional>",
  "properties": [ ... FULL list of properties ... ]
}] }

Strict rules:
- patch.uri must be from the allowlist (never a property URI)
- If you send "properties", it REPLACES the entire list
- connectivity → add property_type "object" towards a class from the allowlist
- structural_integrity → parent_uri for orphan_class towards the root

[User - simulated example]
Allowed class URIs: [ " ... #StudyPlan", " ... #Course", " ... #Competency" ]
Gate failures:
{ "failures": ["connectivity", "structural_integrity"],
  "failure_items": {
    "structural_integrity": [ {"entity_uri": " ... #Competency", "issue": "orphan_class"} ],
    "connectivity": [ {"entity_uri": " ... #Topic3", "issue": "not_linked"} ]
  }
}

Active repairs:
- STRUCTURAL_INTEGRITY: set parent_uri for orphan_class → StudyPlan
- CONNECTIVITY: add object property on Topic3 → range StudyPlan
Return ONLY valid JSON.
```

🔧 surgeon_gate_repairs

Situation	Non-LLM Action
orphan_class	parent_uri → Plan / Guide
not_linked	Add forced object property
no_properties	Add description (xsd:string)
missing Person	Create suggested Person class

Loop 2: A-box Rules and LLM Feedback

▼ A-box Rules () abox_repairs.py

Rule	What it does (before materializing)
URI Normalization	domain#X → canonical URI
Semantic Re-typing	Corrects erroneously assigned type
Chunk Co-occurrence	Same fragment → forces edge
Gates repair	Attempts 2+: uses target label from catalog
Noise filter	Eliminates false positives (stray authors)

- 🔍 **Gates (abox_health.py):** measure JSON + graph written in DB (skipped, dropped_relations, ratio, islands).
- 🚫 **DO NOT reread the PDF:** They measure real data in the database.
- 💬 If they fail → inject explicit feedback block.
- 🔧 Each attempt **RE-EXTRACTS & REWRITES** entirely.

💬 Prompt injection: REFINEMENT FEEDBACK

```
--- REFINEMENT FEEDBACK ---
The previous A-box JSON did not pass quality gates. Address ALL points below:
- Populate object_relations to link individuals. Use property_label from T-box OBJECT properties.
  - Link 'Prof. Ana López' → 'Big Data' via T-box property (target_type "individual").
  - Available object property: 'imparte' (domain 'Profesor').
- Fix relation 'Tema 3' → 'domain#Course' (failed: unknown_class_target). Use exact target labels.
- Fix: 'Introducción al aprendizaje automático' is a topic but was typed as 'Person'
  (class_kind organization) use class_uri 'http://example.org/domain#Asignatura'.
- Every individual MUST use a class_uri that appears in the T-box list. Do not invent new classes.
```